Persistence, Not Variability: When the Speculative Decoding Speedup Formula Over- and Under-Predicts

Tanish Garg Ananya Soni
Rishihood University

Abstract

Speculative decoding lets a small draft model guess γ tokens that a large target model verifies in one pass, without changing the output distribution. The standard analysis of Leviathan, Kalman and Matias assumes a constant acceptance rate α and predicts $(1-\alpha^{\gamma+1})/(1-\alpha)$ tokens per pass. We ask what happens when acceptance varies. We prove a two-sided bound: if difficulty is redrawn every round the formula is a lower bound (Jensen), and if difficulty persists across tokens it is an upper bound, a harmonic mean, because rounds start where guesses fail. Variability with no persistence changes nothing. We also show that the natural adaptive rule, draft while the survival probability exceeds a threshold, is throughput-optimal when rounds are independent but not when acceptance persists, where it over-drafts in easy text and loses up to 3.8% to the exact optimum in a two-regime model. To test this on real models we give a replay method that is exact in distribution on text written by the target model. On Qwen2.5 (0.5B drafting for 3B), the formula overestimates tokens per pass by 2–6% under sampling and by up to 9% under greedy decoding; on Spec-Bench the overestimate is 1–3% for five of six tasks. Replaying reference text instead, as is common, understates persistence. Measured on a T4 GPU, one draft step costs 0.66 of a target step and verification costs 1.2, so the best fixed γ gives at most a 1.09× speedup. A calibrated threshold rule gains 5–10% over the best fixed γ under sampling when drafting is cheap, and 8–12% at the measured costs under greedy decoding.

1 Introduction

A large language model writes one token per forward pass. Speculative decoding [1, 2] speeds this up with a second, smaller model: the draft model M_q guesses γ tokens, the target model M_p scores all of them in one pass, and a rejection rule keeps the output distributed exactly as the target's. Leviathan et al. analyse the method under one assumption, that each guess is accepted independently with the same probability α, and obtain the expected number of tokens per pass and the speedup as functions of α, γ and the cost ratio c.

Real text is not uniformly predictable. Acceptance differs by task [6], a fixed γ is known to be suboptimal [3], and stopping rules based on predicted acceptance have been proposed and learned [4, 5]. This paper works at the level of the original formulas and asks two questions. Is the tokens-per-pass formula still right when α varies? And is the natural threshold rule for choosing γ on the fly actually optimal?

Contributions.

1. **A two-sided bound** (Propositions 2 and 3). Round-level variation makes the formula pessimistic; token-level persistence makes it optimistic, with a harmonic mean in place of the arithmetic one. The second half rests on an exact identity (Lemma 2) that proves $1/g$ convex for every γ.
2. **The scope of the threshold rule** (Proposition 4, Section 4.2). The rule "draft while survival probability $\geq \lambda^* c$" is throughput-optimal for independent rounds. Under Markov persistence we compute the exact optimum and show the rule is not optimal, and that no single threshold can be.
3. **An exact replay protocol** (Lemma 3). Acceptance can be replayed along text written by the target with per-token probability $\min(1, q(x_t)/p(x_t))$, which reproduces speculative decoding in distribution for every γ from one pass of each model.
4. **Measurements** on Qwen2.5 models over prose, maths, code and Spec-Bench, under sampling and greedy decoding, with measured draft and verification costs and confidence intervals.

We also re-derive the exactness proof and the best fixed γ. The closed form for the optimal γ through the Lambert W function is already known [8, 9]; we include a short derivation for completeness and do not claim it.

2 Background and notation

Let p and q be the next-token distributions of M_p and M_q at a position. One *round* drafts γ tokens with M_q, verifies them with one pass of M_p, accepts a prefix, and emits one further token from M_p. Let N be the number of tokens a round produces and T its time in units of one target step. The cost ratio c is the time of one draft step divided by the time of one target step. We write $g_\gamma(\alpha) = \sum_{k=0}^{\gamma} \alpha^k$, and g when γ is fixed.

Lemma 1 (Exactness [1, 2])**.** *Draw $x \sim q$, accept it with probability $\min(1, p(x)/q(x))$, and otherwise draw from $r = \max(0, p-q)/(1-\beta)$, where $\beta = \sum_x \min(p(x), q(x)) = 1 - \frac{1}{2}\|p-q\|_1$. The output is distributed as p, and $\Pr[\mathit{accept}] = \beta$.*

Proof. The output equals x by acceptance with probability $q(x)\min(1, p(x)/q(x)) = \min(p, q)(x)$, or by rejection and resampling with probability $(1-\beta)\, r(x) = \max(0, p-q)(x)$. The two sum to $p(x)$. Since $\sum_x \max(0, p-q) = 1-\beta$, r is a distribution. □

We use the following assumptions.

(A1) *Constant acceptance.* Acceptances in a round are independent, each with probability α.

(A1′) *Round-level variation.* Each round draws its own α (mean $\bar{\alpha}$, variance σ^2) independently of the past; within the round acceptances are independent with that α.

(A1″) *Token-level persistence.* The text consists of long stretches; a stretch at level α makes up a share $w(\alpha)$ of all tokens.

(A2) *Free verification.* One target pass over $\gamma + 1$ positions costs one target step, so $T = \gamma c + 1$. We relax this in Section 6.2 to $T = \gamma c + v$ for $\gamma \geq 1$.

Under (A1), $\Pr[N \geq n+1] = \alpha^n$ for $n \leq \gamma$, so

$$\mathbb{E}[N] = g_\gamma(\alpha) = \frac{1-\alpha^{\gamma+1}}{1-\alpha}, \qquad S(\gamma) = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}. \tag{1}$$

We call (1) the *landmark formula* [1, Theorem 3.8].

Proposition 1 (Best fixed γ)**.** *Under (A1)–(A2): (a) S is quasi-concave in γ. (b) Some $\gamma \geq 1$ beats no speculation if and only if $\alpha > c$. (c) The real maximiser is*

$$\gamma^* = \frac{y - L - c}{Lc}, \qquad L = -\ln \alpha, \qquad y = -c\, W_{-1}\Big(-e^{-1-L(1/c-1)}\Big). \tag{2}$$

Proof. (a) The numerator of S is positive, increasing and concave in γ, and the denominator is positive and affine, so each set $\{S \geq t\}$ is an interval. (b) The "if" part is Corollary 3.9 of [1]: $S(1) = (1+\alpha)/(1+c) > 1$. Conversely, if $\alpha \leq c$ then $\alpha^k \leq \alpha \leq c$ for $k \geq 1$, so $g_\gamma(\alpha) \leq 1 + \gamma c$ and $S(\gamma) \leq 1 = S(0)$. (c) With $u = \alpha^{\gamma+1}$, $S' = 0$ gives $Lu(\gamma c + 1) = c(1-u)$. Putting $y = L(\gamma c+1) + c$ gives $u = c/y$ and $(y/c)\, e^{-y/c} = e^{-1-L(1/c-1)}$, a Lambert-$W$ equation [13]. Since $y > c$ we have $-y/c < -1$, which selects the branch W_{-1}. Equivalent forms appear in [8, 9]. □

In words: speculation is worth trying exactly when a guess is more likely to be accepted than it costs, and the best draft length grows quickly as $\alpha \to 1$. For example $\alpha = 0.8$, $c = 0.05$ gives $\gamma^* = 7.86$; since $S(7) = 3.082 < S(8) = 3.092$, the best integer is 8.

3 When acceptance varies

Proposition 2 (Round-level variation: the formula is a lower bound)**.** *Under (A1′), for every fixed γ,*

$$\mathbb{E}[N] = \sum_{k=0}^{\gamma} \mathbb{E}[\alpha^k] \;\geq\; g_\gamma(\bar{\alpha}), \qquad \mathbb{E}[N] - g_\gamma(\bar{\alpha}) \approx \sigma^2 \sum_{k=2}^{\gamma} \binom{k}{2} \bar{\alpha}^{\,k-2}. \tag{3}$$

Proof. Given α, (1) gives $\mathbb{E}[N \mid \alpha] = g_\gamma(\alpha)$. Average over α and apply Jensen's inequality [14] to the convex maps $x \mapsto x^k$, $k \geq 2$, on $[0, 1]$. The approximation is the second-order Taylor term of each x^k at $\bar{\alpha}$. □

Remark. If instead every token draws its own independent α_j, then $\mathbb{E}[\alpha_1 \cdots \alpha_k] = \bar{\alpha}^{\,k}$ and (1) is exact. Variability alone does nothing; what matters is how long a level of difficulty lasts.

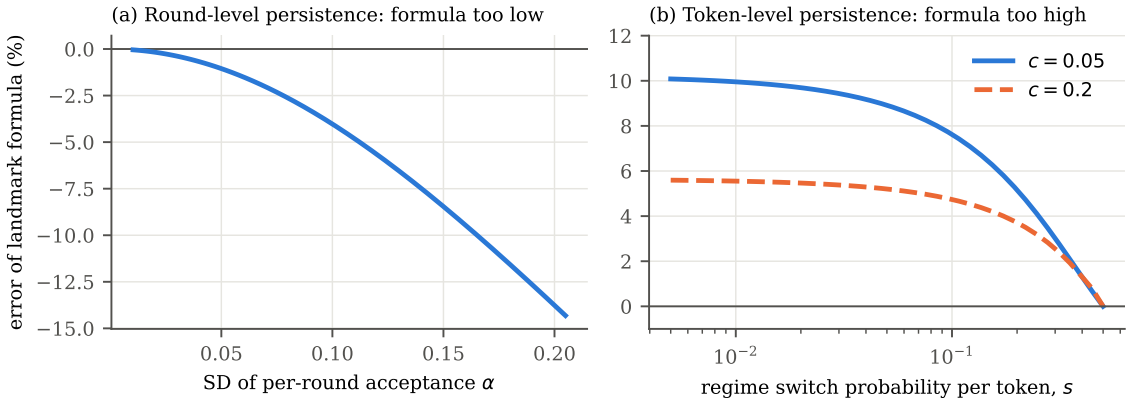


Figure 1: The sign of the formula's error depends on how long difficulty lasts. (a) Model A, $\bar{\alpha} = 0.7$, $c = 0.05$, $\gamma = 6$: acceptance redrawn each round. (b) Model C, $\bar{\alpha} = 0.675$: two regimes that switch per token with probability s; best fixed γ. Both panels are exact computations.

Lemma 2 (Convexity of $1/g$)**.** *For every integer $\gamma \geq 1$ and $g = g_\gamma$,*

$$2\,g'(\alpha)^2 - g(\alpha)\,g''(\alpha) = (\gamma+1)\sum_{s=1}^{\gamma} s\,(\gamma+1-s)\,\alpha^{\gamma+s-2}. \tag{4}$$

Hence $(1/g)'' = (2g'^2 - gg'')/g^3 \geq 0$ and $1/g$ is convex on $[0,1]$.

Proof. Write $g'^2 = \sum_{i,j} ij\,\alpha^{i+j-2}$ and $gg'' = \sum_{i,j} j(j-1)\,\alpha^{i+j-2}$ with $0 \leq i,j \leq \gamma$. The coefficient of α^{n-2} in $2g'^2 - gg''$ is the sum of $2ij - j(j-1)$ over pairs with $i+j=n$. Symmetrising in (i,j) and writing $d = i-j$, so that $ij = (n^2-d^2)/4$ and $i^2+j^2 = (n^2+d^2)/2$, each pair contributes $n^2/4 + n/2 - 3d^2/4$. The admissible values are $d = -m, -m+2, \ldots, m$, with $m = n$ if $n \leq \gamma$ and $m = 2\gamma - n$ if $n > \gamma$. There are $m+1$ of them and $\sum d^2 = m(m+1)(m+2)/3$, so the coefficient equals $\frac{m+1}{4}\,[\,n(n+2) - m(m+2)\,]$. For $n \leq \gamma$ this is 0. For $n > \gamma$, $n(n+2) - m(m+2) = (n-m)(n+m+2) = 4(n-\gamma)(\gamma+1)$, and the coefficient is $(2\gamma - n + 1)(n-\gamma)(\gamma+1) > 0$. Setting $s = n - \gamma$ gives (4). □

We checked (4) symbolically for $\gamma = 1, \ldots, 80$.

Proposition 3 (Token-level persistence: the formula is an upper bound)**.** *Under (A1″) with a fixed γ, the long-run mean number of tokens per round is a harmonic mean, and*

$$\bar{N}_{\text{round}} = \Big(\mathbb{E}_w\big[1/g_\gamma(\alpha)\big]\Big)^{-1} \;\leq\; g_\gamma(\bar{\alpha}) \;\leq\; \mathbb{E}_w\big[g_\gamma(\alpha)\big]. \tag{5}$$

Proof. By the renewal theorem [15], a stretch of ℓ tokens at level α is covered by about $\ell/g_\gamma(\alpha)$ rounds. Rounds therefore fall in level α in proportion to $w(\alpha)/g_\gamma(\alpha)$, and the mean of N per round is $\sum w / \sum (w/g)$. Lemma 2 and Jensen's inequality give $\mathbb{E}_w[1/g] \geq 1/g(\bar{\alpha})$, which is the left inequality. The right one is Proposition 2. □

In words: rounds start right after rejections, and rejections happen in hard text. A randomly chosen round is therefore more often hard than a randomly chosen token, which is the inspection paradox [15]. The formula, fed the token-average $\bar{\alpha}$, is too optimistic.

Two synthetic models. Figure 1 shows both effects by exact computation. In *Model A*, each round draws $\alpha \sim$ Beta with mean 0.7. At $\gamma = 6$ the formula is low by 1.7%, 4.0%, 7.3% and 12.3% for $\mathrm{SD}(\alpha) = 0.064, 0.10, 0.138, 0.187$; the second-order term in (3) predicts 1.7%, 4.1%, 7.6% and 13.1%. In *Model C*, each token is in an easy regime ($\alpha = 0.95$) or a hard one ($\alpha = 0.40$), and the regime switches after each token with probability s, so $\bar{\alpha} = 0.675$. For $c = 0.05$ and $\gamma = 5$ the formula predicts a speedup of 2.229. The exact value is 2.027 at $s = 0.01$, 2.046 at $s = 0.05$ and 2.119 at $s = 0.2$, so the formula is high by 10.0%, 8.9% and 5.2%; the harmonic bound (5) gives 2.022, the limit as $s \to 0$. Half the tokens are hard, but 76%, 75% and 69% of rounds start in hard text. At $s = 0.5$ the regime is independent from token to token and the formula is exact, as the Remark says.

4 Choosing the draft length adaptively

4.1 A threshold rule for independent rounds

Let A_k be the probability that the first k drafts of a round are all accepted, given what the controller knows when the round starts. A_k is non-increasing in k.

Proposition 4 (Threshold rule)**.** *Suppose rounds are independent, as under (A1′), verification is free (A2), and the controller knows $A_1, A_2, \ldots$ before drafting. Then the throughput-optimal policy drafts token k if and only if*

$$A_k \geq \lambda^* c, \qquad \lambda^* = \max_{\pi} \mathbb{E}[N_\pi]/\mathbb{E}[T_\pi]. \tag{6}$$

Proof. By the renewal-reward theorem [15], long-run throughput is $\mathbb{E}[N]/\mathbb{E}[T]$. By Dinkelbach's lemma [12], a policy maximises this ratio if and only if it maximises $\mathbb{E}[N - \lambda^* T]$, which separates by round because rounds are independent. Given the information, $\mathbb{E}[N] = \sum_{k \leq \gamma} A_k$, so adding draft k changes the objective by $A_k - \lambda^* c$. Since A_k is non-increasing, the optimum takes the largest k with $A_k \geq \lambda^* c$. □

In words: $\lambda^* c$ is the opportunity cost of one draft step, measured in tokens. Draft while the chance that the next guess will count is worth more than the time it takes. Under (A1′) the rule is $\gamma(\alpha) = \lfloor \ln(\lambda^* c)/\ln \alpha \rfloor$. In Model A it gains 1.8%, 5.2%, 12.6% and 30.0% over the best fixed γ at the four spreads above. That the optimal stopping policy is a threshold on the rejection probability is shown for an MDP formulation in [4]; our derivation identifies the threshold with the throughput itself.

4.2 The rule is not optimal when acceptance persists

The proof of Proposition 4 uses independence in one place: the objective separates by round. Under persistence, how far a round drafts changes the state in which the next round starts. We tested the rule on Model C, which is a semi-Markov decision problem [16].

Regime observed at the start of each round. With two states and a draft length for each, the throughput of every policy can be computed exactly from the stationary distribution of the round-start regime; we enumerate all policies with at most 40 drafts. Table 1 compares the optimum with rule (6), where A_k accounts correctly for switching and λ is the rule's own throughput. At $s = 0.5$ rounds are independent and the rule is exactly optimal. With persistence it drafts far too much in easy text (35 tokens against 19 at $s = 0.01$, $c = 0.05$) and loses 0.2–2.7%. No single threshold θ can reproduce the optimum: at $s = 0.01$, $c = 0.05$ it would need $A_{20}(\text{easy}) < \theta \leq A_{19}(\text{easy})$, which is near 0.3, together with $\theta \leq A_2(\text{hard}) = 0.16$.

Regime hidden. When the controller sees only accept/reject outcomes, the state is its belief about the regime. We solved the belief problem on a 401-point grid by relative value iteration and scored both policies with the same simulation of 1.5 million rounds. The optimum beats the rule by 3.2%, 2.6% and 1.0% for $s = 0.01, 0.05, 0.2$ at $c = 0.05$, and by 3.8%, 2.0% and 0.3% at $c = 0.2$. Because the grid optimum is itself approximate, these are lower bounds on the rule's loss, up to simulation noise.

Interpretation. The threshold $\lambda^* c$ prices a draft step at the average throughput. Inside an easy stretch the controller is producing tokens much faster than average, so time is worth more there and drafting should stop sooner. Consistent with this, the best fixed γ for a constant $\alpha = 0.95$ at $c = 0.05$ is 21, close to the 19 used by the optimum in the easy regime. We offer this as a reading of the numbers, not a theorem; the practical consequence is that the threshold should depend on the state.

5 Measuring acceptance on real models

5.1 Exact replay on text written by the target

Running speculative decoding once for every γ and every stopping rule is expensive. A common shortcut records the per-position acceptance probability β_t along a reference text and replays rounds with independent Bernoulli(β_t) draws. This has two problems. Reference text is not distributed as the target's own output, which is what speculative decoding conditions on. And β_t averages over the token at position t, although acceptance and the token are dependent: accepted tokens are the ones the draft model likes.

Lemma 3 (Exact replay)**.** *Let $x_1, x_2, \ldots$ be sampled from M_p, and let $a_t = \min\big(1,\, q_t(x_t)/p_t(x_t)\big)$, where p_t, q_t are the two models' distributions given $x_{<t}$. Replaying rounds along this text, accepting the draft at*

Table 1: Model C with the regime observed at the start of each round (exact). Speedups; in brackets the draft length, or (drafts in easy text, drafts in hard text). "Formula error" is the landmark formula against the exact best fixed γ; "hard starts" is the share of rounds that begin in hard text. "Loss" is the optimum over the threshold rule.

c	s	Best fixed (γ)	Formula error (%)	Hard starts (%)	Threshold rule (easy, hard)	Optimum (easy, hard)	Loss (%)
0.05	0.01	2.027 (5)	+10.0	76	2.273 (35, 2)	2.323 (19, 2)	2.2
0.05	0.05	2.046 (5)	+8.9	75	2.257 (22, 2)	2.289 (14, 2)	1.4
0.05	0.2	2.119 (5)	+5.2	69	2.226 (10, 3)	2.231 (9, 3)	0.2
0.05	0.5	2.229 (5)	0.0	50	2.249 (6, 4)	2.249 (6, 4)	0.0
0.2	0.01	1.442 (2)	+5.6	64	1.609 (18, 1)	1.653 (9, 1)	2.7
0.2	0.05	1.447 (2)	+5.2	64	1.601 (12, 1)	1.633 (7, 1)	2.0
0.2	0.2	1.467 (2)	+3.7	61	1.591 (6, 1)	1.602 (5, 1)	0.7
0.2	0.5	1.524 (3)	0.0	50	1.580 (3, 1)	1.580 (3, 1)	0.0

position t with probability a_t independently, has the same law as speculative decoding, for every γ and every stopping rule that depends only on the text so far and on past outcomes.

Proof. At a drafted position, speculative decoding outputs x and accepts with probability $\min(p, q)(x)$, and outputs x after a rejection with probability $\max(0, p - q)(x)$ (Lemma 1). In the replay, $\Pr[x, \text{accept}] = p(x) \min(1, q(x)/p(x)) = \min(p, q)(x)$ and $\Pr[x, \text{reject}] = p(x) - \min(p, q)(x) = \max(0, p - q)(x)$. The token after the last accepted draft is drawn from p in both. The claim follows by induction over positions. □

For greedy decoding, $a_t = 1$ if $\arg\max q_t = x_t$ and 0 otherwise. One sampled continuation and one forward pass of each model therefore suffice for all γ. A stopping rule that reads the draft model's confidence is covered as long as every draft so far was accepted; after a rejection inside a draft, the real draft model continues on its own text, and we approximate its confidence by the values along the target's text.

5.2 Setup

Models. Draft: Qwen2.5-0.5B (24 layers). Target: Qwen2.5-3B (36 layers) [11]. We use the base models for continuation tasks and the instruct models with the chat template for Spec-Bench. All runs use half precision on one Tesla T4 with Hugging Face Transformers.

Data. Prose: WikiText-2 test passages [19], 64-token prompt. Maths: GSM8K test questions [20]. Code: HumanEval prompts [21]. We take 100 prompts from each, and the target writes 128 tokens by ancestral sampling at temperature 1 (no top-k or top-p) or greedily. Spec-Bench [10]: all 480 prompts, 80 in each of six task types, first turn only for the multi-turn set, 128 sampled tokens.

Replay and statistics. Each passage is replayed 20–40 times with common random numbers across policies; only complete rounds are counted. We report 95% bootstrap intervals over passages. For the formula's error the bootstrap is paired: each resample recomputes both the pooled acceptance $\bar{\alpha}$ and the replayed tokens per round.

Check against real decoding. We ran speculative decoding itself on the first 30 prompts of each continuation task under sampling, and compared tokens per round with the exact replay on the same prompts. For $\gamma = 3$ and 6: prose 2.50 against 2.70 and 3.30 against 3.41; maths 3.08 against 3.02 and 3.82 against 4.14; code 3.15 against 3.15 and 4.70 against 4.42. The 95% intervals overlap in all six cases and the differences have both signs, but they reach 8%, and the two sides use independently sampled text. We regard the replay as supported, not as tightly validated.

Reference text. Table 2 compares acceptance along the target's own text with acceptance along the reference continuations of the same prompts. Reference text misstates the acceptance level in a direction that depends on the task, and it understates persistence in all three. In an earlier pilot with GPT-2 models [17, 18] replayed on reference text, the formula's error was 0.4–1.4%, which led us to the present protocol.

Table 2: Own text against reference text (sampling, 100 passages; 82 for code reference). Autocorrelation of β_t at lags 1 and 5. The last column is the landmark formula's error at $\gamma = 6$; on reference text it uses the Bernoulli(β_t) replay.

	Mean acceptance		Autocorrelation, lag 1 / 5		Formula error, $\gamma = 6$ (%)	
	Own	Reference	Own	Reference	Own	Reference
Prose	0.741	0.661	0.40 / 0.21	0.22 / 0.07	+4.7	+0.8
Maths	0.829	0.880	0.35 / 0.16	0.26 / 0.11	+4.6	+4.1
Code	0.842	0.902	0.37 / 0.19	0.20 / 0.11	+4.1	+0.7

Table 3: Landmark formula against exact replay on text written by the target (Qwen2.5-0.5B → 3B). SD and autocorrelation (lags 1 and 5) of the per-position acceptance β_t. Error is formula over replay minus one, in per cent, with a paired 95% bootstrap interval over passages.

	n	$\bar{\alpha}$	SD	Autocorr.	Error, $\gamma = 3$	Error, $\gamma = 6$	Error, $\gamma = 10$
Sampling ($T = 1$), base models							
Prose	100	0.741	0.23	0.40 / 0.21	+2.9 [+2.1, +3.6]	+4.7 [+3.3, +5.9]	+4.7 [+3.1, +6.4]
Maths	100	0.829	0.22	0.35 / 0.16	+2.4 [+1.8, +3.1]	+4.6 [+3.5, +5.7]	+4.6 [+3.0, +6.1]
Code	100	0.842	0.18	0.37 / 0.19	+2.2 [+1.5, +2.9]	+4.1 [+2.9, +5.4]	+5.5 [+3.9, +7.3]
Greedy, base models							
Prose	100	0.828	0.38	0.14 / 0.11	+2.7 [+2.1, +3.2]	+6.4 [+5.5, +7.1]	+9.4 [+8.1, +10.4]
Maths	100	0.894	0.31	0.09 / 0.02	+1.7 [+1.0, +2.2]	+2.0 [+1.1, +2.9]	+4.1 [+2.7, +5.3]
Code	100	0.926	0.26	0.18 / 0.08	+2.0 [+1.4, +2.6]	+4.7 [+3.6, +5.8]	+7.6 [+6.2, +9.0]
Spec-Bench, sampling, instruct models							
Multi-turn chat	80	0.757	0.25	0.33 / 0.22	+2.3 [+1.4, +3.2]	+3.4 [+1.9, +4.8]	+3.2 [+1.4, +5.1]
Translation	80	0.647	0.31	0.16 / 0.07	+1.6 [+0.8, +2.4]	+2.0 [+0.9, +3.1]	+1.0 [−0.4, +2.4]
Summarisation	80	0.699	0.26	0.20 / 0.04	+1.6 [+0.7, +2.4]	+2.1 [+0.8, +3.4]	+1.3 [−0.4, +2.9]
Question answering	80	0.656	0.28	0.18 / 0.06	0.0 [−0.8, +0.7]	−0.3 [−1.6, +0.9]	−1.2 [−2.7, +0.3]
Maths reasoning	80	0.892	0.18	0.25 / 0.11	+1.1 [+0.5, +1.7]	+1.8 [+0.7, +3.0]	+3.1 [+1.3, +5.0]
Retrieval (RAG)	80	0.710	0.27	0.21 / 0.07	+1.4 [+0.5, +2.3]	+1.7 [+0.4, +3.1]	+0.9 [−0.8, +2.7]

6 Results

6.1 The formula overestimates tokens per pass

Table 3 and Figure 2 compare the landmark formula, evaluated at the measured mean acceptance, with the exact replay. Under sampling the formula is high by 2.2–2.9% at $\gamma = 3$ and by 4.1–5.5% at $\gamma = 6$ and 10, on all three text types, with intervals that exclude zero. Under greedy decoding the error keeps growing with γ and reaches 9.4% for prose. On Spec-Bench the error is smaller: 1.1–2.3% at $\gamma = 3$ for five of six task types, again excluding zero, and indistinguishable from zero for question answering. The sign is the one Proposition 3 predicts, and nowhere do we see the underestimate of Proposition 2.

The error tracks persistence only loosely. Spec-Bench tasks have lower lag-1 autocorrelation (0.16–0.33) than the continuation tasks under sampling (0.35–0.40) and smaller errors. Greedy decoding, however, has low lag-1 autocorrelation and the largest errors. Part of the persistence sits between passages: applying the harmonic mean (5) to passage-level acceptance alone accounts for 19–24% of the $\gamma = 6$ error under sampling and 20–37% under greedy decoding. The rest comes from structure within passages that lag-1 autocorrelation does not summarise.

6.2 What a round really costs

We timed both models on the T4 with a key-value cache at context lengths 128 and 256 (medians of 50 runs). One draft step takes 25.6 ms and one target step 38.9 ms, so $c = 0.66$. Leviathan et al. report $c < 0.05$ in their experiments. Our two models differ sixfold in parameters, but the time per layer is almost the same, 1.07 ms for the draft and 1.08 ms for the target, which suggests that at this scale and batch size the latency is set by per-layer overhead in the software stack and not by arithmetic. We did not profile this further, and c would be different in an optimised serving system.

Verifying $m \geq 2$ tokens in one pass costs $v \approx 1.2$ target steps (median 1.21 over $m = 2, \ldots, 13$ and both context lengths, range 1.17–1.50, the high values having wide spreads). The cost is flat in m. We therefore

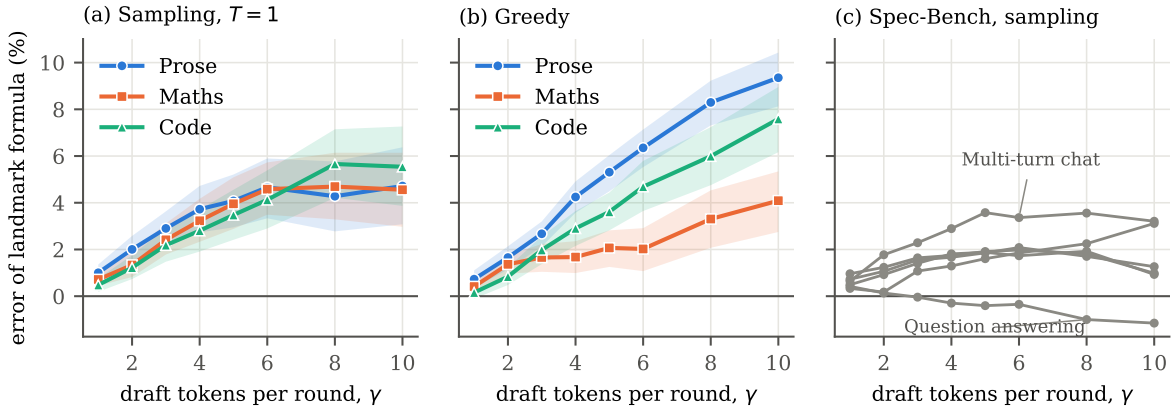


Figure 2: Error of the landmark formula against exact replay, by draft length. Bands are paired 95% bootstrap intervals. In (c) each grey line is one Spec-Bench task type; the highest and lowest are labelled.

use $T = \gamma c + v$ with $v = 1.2$ for $\gamma \geq 1$, and $T = 1$ for $\gamma = 0$.

With these costs the condition of Proposition 1(b) tightens. A single draft helps only if $\alpha > c+v-1 = 0.86$, and some $\gamma \geq 1$ helps only if $\alpha > 0.83$ under (A1). Mean acceptance in our runs is 0.65–0.93. The best fixed γ accordingly gives no speedup under sampling for prose, maths, code and five Spec-Bench tasks ($\gamma = 0$ is chosen or ties), 1.06× for Spec-Bench maths reasoning, and 1.00–1.09× under greedy decoding (Table 4, right).

6.3 Adaptive rules

We compare, by exact replay with two-fold cross-fitting (every parameter is tuned on one half of the passages and scored on the other):

- *Best fixed* γ.
- *Heuristic*: γ starts at 5, rises by 2 after a fully accepted round and falls by 1 otherwise, modelled on the heuristic schedule in Hugging Face assisted generation.
- *Per-token cutoff*: stop after the first draft token whose predicted acceptance is below τ, in the spirit of [3].
- *Calibrated threshold rule*: rule (6) with A_k estimated as the running product of predicted acceptances, in the spirit of [4].
- *Oracle*: rule (6) with the true β_t known before drafting. Under greedy decoding this would amount to knowing the outcomes, so we omit it.

Predicted acceptance comes from a calibrator fitted to the draft model's top probability and entropy (isotonic or logistic regression, whichever fits the training half better). Both confidence-based rules pay for every draft step they run, including the one whose confidence triggers the stop, and keep that token. The per-token cutoff and the calibrator are simplified stand-ins, not the trained classifiers of [3, 4], so the comparison is between stopping criteria, not between published systems.

Cheap drafting (Table 4, left). The calibrated threshold rule gains 4.7–10.1% over the best fixed γ in all nine sampling conditions and beats the per-token cutoff in each. It recovers 38–79% of the oracle's gain of 10–14%. Under greedy decoding both confidence rules gain more, 10–19%, and are within about three points of each other. The heuristic loses to the best fixed γ under sampling and is mixed under greedy decoding. The gains are stable for $c = 0.1$ and 0.3.

Measured costs (right). Under greedy decoding the confidence rules reach 1.08–1.23× where the best fixed γ gives 1.00–1.09×, a gain of 8–12%. Under sampling the picture is mixed: they help on maths, code and Spec-Bench maths reasoning (1.05–1.14×) and are slower than not speculating on the other six conditions (0.88–0.98×). The reason is structural. A confidence rule must run the draft model at least once per round to read its confidence, and at $c = 0.66$ that look is expensive. The oracle, which decides before drafting, still gains 6–21% over plain decoding in every sampling condition. A rule that decides *whether* to speculate before running the draft model is the missing piece.

Table 4: Adaptive rules by exact replay. Left: assumed costs $c = 0.2$, $v = 1$; speedup of the best fixed γ over plain decoding, then gain of each rule over the best fixed γ in per cent. Right: measured costs $c = 0.66$, $v = 1.2$; speedups over plain decoding. Intervals are in the supplementary result files; every left-hand gain of the threshold rule has a 95% interval that excludes zero.

	$c = 0.2$, free verification					Measured: $c = 0.66$, $v = 1.2$			
	Fixed (×)	Heur. (%)	Cutoff (%)	Thresh. (%)	Oracle (%)	Fixed (×)	Cutoff (×)	Thresh. (×)	Oracle (×)
Sampling ($T = 1$), base models									
Prose	1.64	−3.4	+1.9	+6.1	+9.9	1.00	0.96	0.96	1.08
Maths	1.91	−2.4	+5.4	+8.5	+13.8	1.00	1.05	1.05	1.14
Code	1.96	−2.2	+2.9	+7.9	+11.6	1.00	1.05	1.06	1.13
Greedy, base models									
Prose	1.89	+2.8	+14.6	+15.9	–	1.00	1.08	1.08	–
Maths	2.27	−2.8	+9.9	+12.9	–	1.05	1.16	1.16	–
Code	2.44	+4.2	+18.9	+18.8	–	1.09	1.23	1.22	–
Spec-Bench, sampling, instruct models									
Multi-turn chat	1.69	−2.8	+4.7	+8.1	+11.6	1.00	0.98	0.98	1.10
Translation	1.46	−5.0	+3.5	+4.7	+12.4	1.00	0.88	0.88	1.06
Summarisation	1.55	−5.2	+4.2	+5.5	+10.4	1.00	0.93	0.93	1.07
Question answering	1.49	−5.3	+3.6	+4.9	+10.4	1.00	0.90	0.90	1.06
Maths reasoning	2.27	−0.9	+5.4	+10.1	+12.8	1.06	1.14	1.14	1.21
Retrieval (RAG)	1.58	−4.9	+4.4	+6.7	+11.4	1.00	0.94	0.94	1.08

7 Related work

Speculative decoding was introduced in [1, 2]. Adaptive draft lengths have been studied through confidence classifiers [3], a threshold on the predicted rejection probability derived from an MDP [4], and reinforcement learning [5]. Closed forms for the optimal fixed length appear in [8], with a communication cost, and in [9]. Acceptance has been shown to differ across task domains [6], and verification has been measured to be far from free on consumer hardware [7], which our T4 timings echo. Spec-Bench [10] standardises evaluation. To our knowledge, the two-sided bound, the identity behind it, the failure of a single threshold under persistence and the exact replay identity are new; we would welcome pointers to earlier statements.

8 Limitations

- **One model pair, small models, one GPU.** All measurements use Qwen2.5-0.5B and 3B on a T4 with a general-purpose library. The cost ratio $c = 0.66$ belongs to this stack.
- **Replay validation is loose.** The check against real decoding uses 30 prompts and agrees only to within 8%.
- **Adaptive rules are replayed, not deployed.** Speedups come from the cost model $T = \gamma c + v$ with measured constants, not from wall-clock runs of the rules, and use an approximation after rejections inside a draft.
- **Idealised theory.** (A1′) and (A1″) are extremes; Proposition 3 assumes long stretches; the explanation of why the threshold rule over-drafts is an interpretation.
- **Simplified baselines.** We did not run the published implementations of [3, 4].

9 Conclusion

The speculative decoding speedup formula is exact when acceptance has no memory, too low when difficulty is redrawn each round, and too high when it persists from token to token. On real models we only see the last case: the formula overestimates tokens per pass by a few per cent, more under greedy decoding. The threshold rule for adaptive drafting is optimal only for independent rounds; with persistence the right threshold depends on the state. And on a commodity GPU the cost of the draft model, not the acceptance rate, decides whether speculation pays. Open problems are a state-dependent rule with guarantees under Markov acceptance, and a cheap signal for deciding whether to draft at all.

Acknowledgements. [To be completed. Note for the authors: most venues require a statement of how AI tools were used in the research and writing; add it here.]

References

[1] Y. Leviathan, M. Kalman and Y. Matias, "Fast inference from Transformers via speculative decoding," in *Proc. 40th Int. Conf. on Machine Learning (ICML)*, PMLR vol. 202, pp. 19274–19286, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[3] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat and R. Schwartz, "Dynamic speculation lookahead accelerates speculative decoding of large language models," in *Proc. 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, PMLR vol. 262, pp. 456–467, 2024.

[4] K. Huang, X. Guo and M. Wang, "SpecDec++: Boosting speculative decoding via adaptive candidate lengths," in *Conf. on Language Modeling (COLM)*, 2025. arXiv:2405.19715.

[5] J. Zhang et al., "Learning to draft: Adaptive speculative decoding with reinforcement learning," in *Int. Conf. on Learning Representations (ICLR)*, 2026. arXiv:2603.01639.

[6] S. Abdellatif, "Acceptance dynamics across cognitive domains in speculative decoding," arXiv:2604.14682, 2026.

[7] P. Chordiya, "Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware," arXiv:2607.17283, 2026.

[8] C. Zheng and T. Yang, "Communication-efficient collaborative LLM inference via distributed speculative decoding," arXiv:2509.04576, 2025.

[9] A. Bozorgkhoo and I. Molybog, "Speculative decoding scaling laws (SDSL)," arXiv:2603.11053, 2026.

[10] H. Xia et al., "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics: ACL*, 2024.

[11] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[12] W. Dinkelbach, "On nonlinear fractional programming," *Management Science*, vol. 13, no. 7, pp. 492–498, 1967.

[13] R. M. Corless, G. H. Gonnet, D. E. G. Hare, D. J. Jeffrey and D. E. Knuth, "On the Lambert W function," *Advances in Computational Mathematics*, vol. 5, pp. 329–359, 1996.

[14] J. L. W. V. Jensen, "Sur les fonctions convexes et les inégalités entre les valeurs moyennes," *Acta Mathematica*, vol. 30, pp. 175–193, 1906.

[15] S. M. Ross, *Stochastic Processes*, 2nd ed., Wiley, 1996.

[16] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.

[17] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI technical report, 2019.

[18] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[19] S. Merity, C. Xiong, J. Bradbury and R. Socher, "Pointer sentinel mixture models," in *Int. Conf. on Learning Representations (ICLR)*, 2017.

[20] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[21] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.